CSE513 Assignment 1 - CPU Scheduling Simulator

Public website URL: PENDING OWNER-AUTHENTICATED STREAMLIT COMMUNITY CLOUD DEPLOYMENT

The source code has been pushed to the repository. The public Streamlit deployment is still pending. This draft must be updated with the tested public address before submission.

Prepared: 25 October 2026

Implementation: Python 3.11, Streamlit, Plotly

Repository: `https://github.com/MesutHakanTaskiner/GTU-OS-Project`

1. Objective and model

I developed this project to compare four CPU scheduling policies: FCFS, non-preemptive SJF, preemptive SRTF, and Round Robin. The main aim is to make each scheduling decision visible and to calculate the resulting performance measures correctly.

The model contains exactly one CPU, one CPU burst per process, no I/O, and zero-time context switches. Arrival and execution times are integers. Each timeline interval is half-open: `[start,end)`. The CPU cannot idle while a ready process exists. Process identifiers are unique, and the table row order is retained as immutable `input_index`.

This simulator models scheduling policies. It does **not** reproduce the actual Ubuntu scheduler.

I checked the completed work against the four-page assignment specification. I found no conflict that required a change to the algorithms or the numerical results.

2. Algorithms and deterministic tie rules

- **FCFS:** choose the earliest-arriving ready process and run it to completion.
- **SJF:** when the CPU is free, choose the ready process with the smallest original burst and run it non-preemptively. A future short job is never selected early.
- **SRTF:** choose the smallest remaining burst. Preempt a running process only when another ready process is strictly shorter. Equal remaining time keeps the current process.
- **Round Robin:** use a FIFO ready queue. New arrivals append at the tail. At a quantum boundary, arrivals at that exact time are enqueued before the unfinished running process. Each slice remains a distinct timeline segment.

When a new selection has an otherwise equal policy key, earlier arrival wins, followed by earlier original input index. Initial and intermediate idle gaps are explicit `IDLE` segments.

3. Implementation architecture

The scheduling code is kept separate from the Streamlit interface. The `simulate(processes, algorithm, quantum=None)` function validates the input, calls the selected policy, and calculates the metrics from the resulting timeline.

Streamlit is responsible for the editable table and result display, while Plotly draws the Gantt chart. A separate verification function checks CPU-time accounting, arrival times, overlap, nonnegative metrics, and unnecessary idle time. Keeping these responsibilities separate made the scheduling functions easier to test.

Program flow: input table -> validation -> selected scheduler -> timeline -> metrics and checks -> web display.

4. Required worked examples

The following hand results were derived before consulting simulator output. `C` is completion, `T=C-arrival`, `W=T-burst`, and `R=first_start-arrival`.

4.1 Workload A input

Process	Arrival	Burst
P1	0	8
P2	1	4
P3	2	2
P4	4	1

4.2 Workload A - FCFS

Input: Workload A from Section 4.1.

Scheduling steps: at t=0 only P1 is ready, so it runs through t=8. P2, P3, and P4 arrive while P1 runs. Their arrival order is P2, P3, P4, so they execute in that order.

Hand-calculated timeline: `P1[0,8) -> P2[8,12) -> P3[12,14) -> P4[14,15)`.

Hand-calculated metrics:

Process	Arrival	Burst	First start	C	T	W	R
P1	0	8	0	8	8	0	0
P2	1	4	8	12	11	7	7
P3	2	2	12	14	12	10	10
P4	4	1	14	15	11	10	10

Hand calculation: `T=8+11+12+11=42`, `W=0+7+10+10=27`, `R=0+7+10+10=27`. The averages are T=10.50, W=6.75, and R=6.75.

Simulator output: `P1[0,8) -> P2[8,12) -> P3[12,14) -> P4[14,15)`; averages T=10.50, W=6.75, R=6.75. The simulator produced the same first-start, completion, turnaround, waiting, and response values as the hand table.

Comparison: no discrepancy.

4.3 Workload A - SJF

Input: Workload A from Section 4.1.

Scheduling steps: P1 is the only ready process at t=0 and SJF is non-preemptive, so P1 runs to t=8. All other processes are then ready. Their burst order is P4(1), P3(2), P2(4).

Hand-calculated timeline: `P1[0,8) -> P4[8,9) -> P3[9,11) -> P2[11,15)`.

Hand-calculated metrics:

Process	Arrival	Burst	First start	C	T	W	R
P1	0	8	0	8	8	0	0
P2	1	4	11	15	14	10	10
P3	2	2	9	11	9	7	7
P4	4	1	8	9	5	4	4

Hand calculation: `T=8+14+9+5=36`, `W=0+10+7+4=21`, `R=0+10+7+4=21`. The averages are T=9.00, W=5.25, and R=5.25.

Simulator output: `P1[0,8) -> P4[8,9) -> P3[9,11) -> P2[11,15)`; averages T=9.00, W=5.25, R=5.25. The per-process simulator metrics were identical to the hand table.

Comparison: no discrepancy.

4.4 Workload A - SRTF

Input: Workload A from Section 4.1.

Scheduling steps: P1 starts at t=0. At t=1, P2(4) is strictly shorter than P1's remaining 7, so P2 preempts. At t=2, P3(2) is strictly shorter than P2's remaining 3, so P3 preempts and finishes at t=4. P4(1) is then shortest and finishes at t=5. P2(3) runs to t=8, followed by P1(7) to t=15.

Hand-calculated timeline: `P1[0,1) -> P2[1,2) -> P3[2,4) -> P4[4,5) -> P2[5,8) -> P1[8,15)`.

Hand-calculated metrics:

Process	Arrival	Burst	First start	C	T	W	R
P1	0	8	0	15	15	7	0
P2	1	4	1	8	7	3	0
P3	2	2	2	4	2	0	0
P4	4	1	4	5	1	0	0

Hand calculation: `T=15+7+2+1=25`, `W=7+3+0+0=10`, `R=0`. The averages are T=6.25, W=2.50, and R=0.00.

Simulator output: `P1[0,1) -> P2[1,2) -> P3[2,4) -> P4[4,5) -> P2[5,8) -> P1[8,15)`; averages T=6.25, W=2.50, R=0.00. All per-process metrics matched the hand calculation.

Comparison: no discrepancy.

4.5 Workload A - Round Robin, q=2

Input: Workload A from Section 4.1; quantum=2.

Scheduling steps: P1 runs 0-2. P2 arrives at 1 and P3 at the boundary t=2; both enter before P1 is re-enqueued, giving P2,P3,P1. P2 runs 2-4. P4 arrives at boundary t=4 before P2 re-enters, giving P3,P1,P4,P2. Subsequent slices follow FIFO. P1's final two adjacent slices remain distinct.

Hand-calculated timeline: `P1[0,2) -> P2[2,4) -> P3[4,6) -> P1[6,8) -> P4[8,9) -> P2[9,11) -> P1[11,13) -> P1[13,15)`.

Hand-calculated metrics:

Process	Arrival	Burst	First start	C	T	W	R
P1	0	8	0	15	15	7	0
P2	1	4	2	11	10	6	1
P3	2	2	4	6	4	2	2
P4	4	1	8	9	5	4	4

Hand calculation: `T=15+10+4+5=34`, `W=7+6+2+4=19`, `R=0+1+2+4=7`. The averages are T=8.50, W=4.75, and R=1.75.

Simulator output: `P1[0,2) -> P2[2,4) -> P3[4,6) -> P1[6,8) -> P4[8,9) -> P2[9,11) -> P1[11,13) -> P1[13,15)`; averages T=8.50, W=4.75, R=1.75. The two final P1 slices remained separate in the simulator output.

Comparison: no discrepancy.

4.6 Workload B input

Process	Arrival	Burst
P1	2	3
P2	2	1
P3	8	2
P4	9	1

4.7 Workload B - FCFS

Input: Workload B from Section 4.6.

Scheduling steps: no process exists until t=2. P1 and P2 arrive together, so input order selects P1 then P2. Both finish at t=6; the CPU idles until P3 arrives at 8. P3 runs non-preemptively, then P4.

Hand-calculated timeline: `IDLE[0,2) -> P1[2,5) -> P2[5,6) -> IDLE[6,8) -> P3[8,10) -> P4[10,11)`.

Hand-calculated metrics:

Process	Arrival	Burst	First start	C	T	W	R
P1	2	3	2	5	3	0	0
P2	2	1	5	6	4	3	3
P3	8	2	8	10	2	0	0
P4	9	1	10	11	2	1	1

Hand calculation: T=3+4+2+2=11, W=0+3+0+1=4, R=0+3+0+1=4. The averages are T=2.75, W=1.00, and R=1.00.

Simulator output: `IDLE[0,2) -> P1[2,5) -> P2[5,6) -> IDLE[6,8) -> P3[8,10) -> P4[10,11)`; averages T=2.75, W=1.00, R=1.00. The simulator metrics matched the hand table.

Comparison: no discrepancy.

4.8 Workload B - SJF

Input: Workload B from Section 4.6.

Scheduling steps: after IDLE 0-2, both P1(3) and P2(1) are ready, so P2 runs first, followed by P1. The CPU idles 6-8. P3 starts at 8; P4 arrives at 9, but non-preemptive SJF lets P3 finish before P4.

Hand-calculated timeline: `IDLE[0,2) -> P2[2,3) -> P1[3,6) -> IDLE[6,8) -> P3[8,10) -> P4[10,11)`.

Hand-calculated metrics:

Process	Arrival	Burst	First start	C	T	W	R
P1	2	3	3	6	4	1	1
P2	2	1	2	3	1	0	0
P3	8	2	8	10	2	0	0
P4	9	1	10	11	2	1	1

Hand calculation: T=4+1+2+2=9, W=1+0+0+1=2, R=1+0+0+1=2. The averages are T=2.25, W=0.50, and R=0.50.

Simulator output: `IDLE[0,2) -> P2[2,3) -> P1[3,6) -> IDLE[6,8) -> P3[8,10) -> P4[10,11)`; averages T=2.25, W=0.50, R=0.50. The per-process output matched the hand table.

Comparison: no discrepancy.

4.9 Workload B - SRTF

Input: Workload B from Section 4.6.

Scheduling steps through t=8 match SJF. P3 starts at 8. At t=9, P3 has one unit remaining and P4 arrives with one unit. Because equality cannot preempt the running process, P3 keeps the CPU until t=10; P4 then runs.

Hand-calculated timeline: `IDLE[0,2) -> P2[2,3) -> P1[3,6) -> IDLE[6,8) -> P3[8,10) -> P4[10,11)`.

Hand-calculated metrics:

Process	Arrival	Burst	First start	C	T	W	R
P1	2	3	3	6	4	1	1
P2	2	1	2	3	1	0	0
P3	8	2	8	10	2	0	0
P4	9	1	10	11	2	1	1

Hand calculation: the sums are T=9, W=2, and R=2, so the averages are T=2.25, W=0.50, and R=0.50.

Simulator output: `IDLE[0,2) -> P2[2,3) -> P1[3,6) -> IDLE[6,8) -> P3[8,10) -> P4[10,11)`; averages T=2.25, W=0.50, R=0.50. P3 remained one continuous 8-10 segment because the equal remaining time at t=9 did not cause preemption.

Comparison: no discrepancy.

4.10 Workload B - Round Robin, q=2

Input: Workload B from Section 4.6; quantum=2.

Scheduling steps: after IDLE 0-2, P1 enters before P2 by input order. P1 uses a two-unit slice, P2 runs one unit and completes, then P1 completes its final unit. The CPU idles 6-8. P3 completes within one slice while P4 waits, then P4 runs.

Hand-calculated timeline: `IDLE[0,2) -> P1[2,4) -> P2[4,5) -> P1[5,6) -> IDLE[6,8) -> P3[8,10) -> P4[10,11)`.

Hand-calculated metrics:

Process	Arrival	Burst	First start	C	T	W	R
P1	2	3	2	6	4	1	0
P2	2	1	4	5	3	2	2
P3	8	2	8	10	2	0	0
P4	9	1	10	11	2	1	1

Hand calculation: `T=4+3+2+2=11`, `W=1+2+0+1=4`, `R=0+2+0+1=3`. The averages are T=2.75, W=1.00, and R=0.75.

Simulator output: `IDLE[0,2) -> P1[2,4) -> P2[4,5) -> P1[5,6) -> IDLE[6,8) -> P3[8,10) -> P4[10,11)`; averages T=2.75, W=1.00, R=0.75. The per-process simulator values matched the hand table.

Comparison: no discrepancy.

5. Student-designed targeted cases

5.1 Initial and intermediate idle time

Input: I1=(arrival 2, burst 2), I2=(arrival 6, burst 1), tested with FCFS. Expected and actual timeline: `IDLE[0,2) -> I1[2,4) -> IDLE[4,6) -> I2[6,7)`. I1 has T=2,W=0,R=0; I2 has T=1,W=0,R=0. This verifies both idle forms and confirms that idle time is represented rather than skipped.

5.2 Simultaneous arrivals and equal bursts

Input order: T1=(0,3), T2=(0,1), T3=(0,1), tested with SJF. Expected and actual timeline: `T2[0,1) -> T3[1,2) -> T1[2,5)`. T1 has T=5,W=2,R=2; T2 has T=1,W=0,R=0; T3 has T=2,W=1,R=1. This proves that equal shortest bursts use original input order.

5.3 SRTF strict and equal-remaining behavior

Input: E1=(0,5), E2=(1,4), E3=(2,1). At t=1, E1 has four units left, equal to E2, so E1 must continue. At t=2, E1 has three left and E3 has one, so E3 must preempt. Expected and actual timeline: `E1[0,2) -> E3[2,3) -> E1[3,6) -> E2[6,10)`. Metrics: E1 T=6,W=1,R=0; E2 T=9,W=5,R=5; E3 T=1,W=0,R=0.

5.4 Additional RR boundary regression

Input B1=(0,4), B2=(2,1), q=2. Expected and actual timeline: `B1[0,2) -> B2[2,3) -> B1[3,5)`. This proves that B2's boundary arrival is enqueued before unfinished B1. A separate single-process test confirms that `Solo[0,2)`, `Solo[2,4)`, and `Solo[4,5)` remain three distinct slices.

All targeted cases passed the five independent invariants.

6. Analysis questions

6.1 Minimum average waiting and response for Workload A

SRTF is minimum for both measures among the four required runs: average waiting 2.50 and average response 0.00. It immediately starts P1 at t=0 and each later arrival P2, P3, and P4 at its arrival because each is strictly shorter than the current remainder. FCFS averages 6.75/6.75, SJF 5.25/5.25, and RR q=2 4.75/1.75 for waiting/response. SRTF moves short jobs forward without delaying their first service, at the cost of completing long P1 last.

6.2 RR quantum 1, 2, and 4 on Workload A

Quantum	Avg. turnaround	Avg. waiting	Avg. response
1	8.50	4.75	0.75
2	8.50	4.75	1.75
4	9.25	5.50	3.75

Quantum 1 improves average response because P2, P3, and P4 receive their first turn sooner. It does not improve turnaround or waiting relative to q=2: extra rotation changes when partial work occurs but yields the same completion-based averages here. Smaller quantum therefore does not uniformly improve metrics. In a real system it would also increase context-switch overhead, although this simulator assigns context switches zero time.

6.3 When SJF and SRTF are identical

They are identical when no arrival during a running job has a burst strictly shorter than that job's remaining time, and both select the same ready job whenever the CPU becomes free. Workload B is a concrete example. At t=9, P4 arrives with burst 1 while P3 has remaining 1; equality does not preempt, so both policies produce P3[8,10) then P4[10,11).

6.4 Why real systems estimate CPU bursts

This simulator receives the entire burst as input, which makes SJF/SRTF selection exact. A real operating system cannot know how long a program will compute before it blocks, exits, or is interrupted; that behavior depends on data, branches, cache effects, and external events. Practical schedulers therefore use history, feedback, priorities, fairness, and estimators rather than an oracle for the upcoming CPU burst.

6.5 Starvation

SJF can postpone a long ready job whenever shorter jobs are available at each selection point. SRTF can repeatedly preempt a long job if a sustained stream of shorter remaining workloads arrives. Thus both can starve long jobs in an open-ended system without aging or fairness controls. These supplied finite workloads eventually stop receiving arrivals, so every process completes; finite completion here does not remove the general starvation risk.

6.6 Real implementation mistake and correction

The first Streamlit smoke test used `AppTest.from_file("app.py")` inside `tests/test_app.py`. Streamlit resolves relative paths against the calling test file, so pytest failed with `FileNotFoundError` for `tests/app.py`. The correction was to resolve the repository root explicitly with `Path(__file__).resolve().parents[1] / "app.py"`. Rerunning the suite then passed all 61 tests. This was not hidden or rewritten as a scheduling anecdote; it was the actual failure encountered during development.

7. Validation and test strategy

Validation rejects empty or duplicate IDs, negative arrival, nonpositive burst/quantum, decimal floats, booleans, strings posing as numbers, NaN, malformed rows, unknown algorithms, duplicate input indices, over 100 processes, individual values over 100,000, and total burst over 1,000,000.

The tests use expected timelines and metrics written independently of the scheduling functions. Workloads A and B are checked under every algorithm. Other tests cover SJF non-preemption, the equal-remaining SRTF rule, RR arrivals at quantum boundaries, consecutive RR slices, simultaneous arrivals, completion at an arrival boundary, and initial or intermediate idle periods.

Commands and actual results on Python 3.11.0:

```
python -m pytest -q
61 passed in 1.36s

streamlit run app.py --server.headless true --server.port 8501
GET /_stcore/health -> HTTP 200, ok
GET / -> HTTP 200
```

The Streamlit smoke test ran the default Workload A/FCFS flow and checked the three displayed averages. A local HTTP health request also returned status 200.

8. Interface and usability

The single-page interface provides editable process rows, Workload A and B buttons, three additional examples, an algorithm selector, and an RR quantum field. The result area contains the Gantt chart, the eight required process columns, three averages, the execution order in text form, and a decision trace. If an input is edited after a run, the page warns that the displayed result is out of date.

Idle intervals use grey. Process colors are stable within a run. RR uses white three-pixel borders and per-slice hover labels so adjacent slices never visually disappear into one block. Every timeline boundary appears as an integer tick and guide line.

9. AI assistance disclosure

I used OpenAI Codex while organizing the requirements, implementing and debugging the Python/Streamlit code, preparing tests, and formatting this report. I checked the generated work with independently prepared schedules, 61 automated tests, invariant checks, a Streamlit smoke test, and a local HTTP health check.

One suggested UI test used `AppTest.from_file("app.py")`. The test failed because Streamlit resolved that path relative to the test directory. I corrected it by building the path from `Path(__file__).resolve().parents[1]`. I also corrected a mixed-type Quantum column after an Arrow serialization warning. I kept these changes only after rerunning the tests.

10. Local use and deployment

Create a Python 3.11 virtual environment, install `requirements-dev.txt`, run `python -m pytest -q`, then run `streamlit run app.py`. The production runtime uses only the pinned packages in `requirements.txt`.

The source is available on the configured GitHub repository's `main` branch. The remaining submission step is to deploy `app.py` on Streamlit Community Cloud, make the app public, and test the link without signing in. The verified address must then replace the pending notice on page 1.

11. Limitations and final status

The simulation intentionally excludes I/O bursts, multicore behavior, nonzero context-switch cost, priorities, and the Linux Completely Fair Scheduler. It is a policy simulator, not a kernel emulator. The authoritative professor PDF has been reviewed with no conflicting requirement found. GitHub push is complete, but public Streamlit deployment remains dependent on authenticated owner action; therefore the website URL is correctly marked pending rather than invented.